OPTIMISATION PROBLEM SHEET

This sheet covers the three optimisation lectures. Question 1 looks at gradient descent and Newton's method on simple functions, where we can see exactly what happens. Questions 2 and 3 use Lagrange multipliers and duality. Question 4 asks you to check some of these results in Python. The early parts of each question are short; the later parts ask you to put the pieces together.

1 Gradient descent with learning rate r updates $w \leftarrow w - r f'(w)$ (or $\boldsymbol{w} \leftarrow \boldsymbol{w} - r\nabla f(\boldsymbol{w})$ in several dimensions). In this question we see exactly when it works on quadratic functions.

(a) For $f(w) = \frac{1}{2}aw^2$ with $a > 0$, write down the gradient descent update and show that after t steps $w_t = (1 - ra)^t w_0$.

$f'(w) = aw$, so

$$w_{t+1} = w_t - raw_t = (1 - ra)w_t.$$

Each step multiplies w by the same factor $1 - ra$, so after t steps $w_t = (1 - ra)^t w_0$.

[3 marks]

(b) For which learning rates does $w_t \to 0$ (the minimum)? Which learning rate is best? Describe what happens when $r = 1.5/a$ and when $r = 2.5/a$.

We need $|1 - ra| < 1$, i.e. $0 < r < 2/a$. The best choice is $r = 1/a$, which gives a factor of 0 and reaches the minimum in one step.

- $r = 1.5/a$: the factor is -0.5. Each step overshoots the minimum, so w alternates in sign, but it still shrinks and converges.
- $r = 2.5/a$: the factor is -1.5. Each step overshoots by more than it started from, so w alternates in sign and grows: gradient descent diverges.

[4 marks]

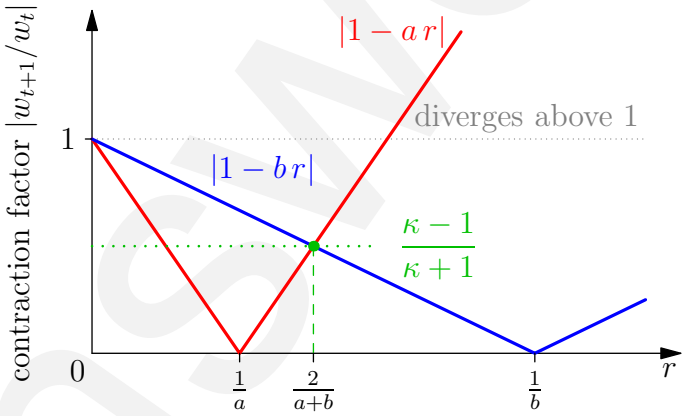
Answers

(c) Now take $f(\boldsymbol{w}) = \frac{1}{2}(aw_1^2 + bw_2^2)$ with $a \geq b > 0$. Show that the two components evolve independently, and find the condition on r for convergence. Show that the fixed learning rate that makes the slower of the two components converge fastest is $r = 2/(a+b)$, and that the error is then multiplied by $(\kappa-1)/(\kappa+1)$ each step, where $\kappa = a/b$ is the condition number.

$\nabla f = (aw_1, bw_2)^\mathsf{T}$, so $w_1 \leftarrow (1-ra)w_1$ and $w_2 \leftarrow (1-rb)w_2$: each component behaves like part (a). Both converge only if $r < 2/a$: the steepest direction limits the step size.

The overall contraction factor (how much the error shrinks each step) is the larger of $|1-ra|$ and $|1-rb|$. As r grows, $|1-rb| = 1-rb$ falls, while for $r > 1/a$, $|1-ra| = ra-1$ rises. The best r balances them:

$$ra - 1 = 1 - rb \quad \Rightarrow \quad r = \frac{2}{a+b}, \qquad 1 - rb = \frac{a-b}{a+b} = \frac{\kappa-1}{\kappa+1}.$$



[5 marks]

(d) Take $a = 100$ and $b = 1$. What is the largest stable learning rate, and the best one? Roughly how many gradient descent steps are needed to reduce the error by a factor of 10^6? How many steps would Newton's method need?

Stability needs $r < 2/a = 0.02$; the best learning rate is $r = 2/101 \approx 0.0198$. Then $\kappa = 100$ and each step multiplies the error by $99/101 \approx 0.980$, so we need

$$n \approx \frac{\ln 10^{-6}}{\ln(99/101)} \approx 690 \text{ steps.}$$

Newton's method takes *one* step. The Hessian is $\mathbf{H} = \mathrm{diag}(100, 1)$, and for a quadratic $\boldsymbol{w} - \mathbf{H}^{-1}\nabla f(\boldsymbol{w}) = \mathbf{0}$ exactly.

[4 marks]

Answers

(e) Newton's method in one dimension updates $w \leftarrow w - f'(w)/f''(w)$. For $f(w) = \mathrm{e}^w - 2w$, show that the minimum is at $w^* = \ln 2$ and that Newton's update is $w \leftarrow w - 1 + 2\mathrm{e}^{-w}$. Starting from $w_0 = 0$, use a calculator to compute four iterations and the error $|w_t - \ln 2|$. What happens to the error?

$f'(w) = \mathrm{e}^w - 2 = 0$ at $w^* = \ln 2$, and $f''(w) = \mathrm{e}^w > 0$ so it is a minimum. Newton's update is $w - (\mathrm{e}^w - 2)/\mathrm{e}^w = w - 1 + 2\mathrm{e}^{-w}$.

t	0	1	2	3	4
w_t	0	1	0.735759	0.694042	0.693148
$\lvert w_t - \ln 2\rvert$	0.69	0.31	4.3×10^{-2}	9.0×10^{-4}	4.0×10^{-7}

Once we are close, the error is roughly *squared* at each step (the number of correct digits doubles): quadratic convergence. The next step gives about 10^{-13}.

[4 marks]

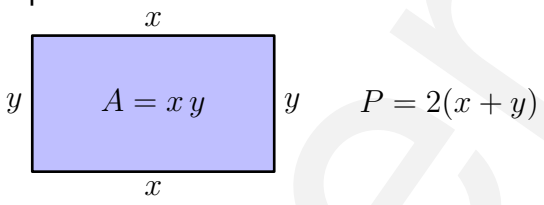
End of question 1

Answers

• Do not write in this space •

2 To minimise (or maximise) $f(\boldsymbol{x})$ subject to $g(\boldsymbol{x}) = 0$ we form the Lagrangian $\mathcal{L}(\boldsymbol{x},\alpha) = f(\boldsymbol{x}) - \alpha g(\boldsymbol{x})$ and set its derivatives with respect to $\boldsymbol{x}$ and α to zero. For an inequality constraint $g(\boldsymbol{x}) \geq 0$ the KKT conditions add $\alpha \geq 0$ and $\alpha g(\boldsymbol{x}) = 0$.

(a) A rectangle has sides x and y and a fixed perimeter $2x+2y = P$. Use a Lagrange multiplier to find the sides that maximise its area $A = xy$. Show that α equals $\mathrm{d}A^*/\mathrm{d}P$, where A^* is the maximum area: the Lagrange multiplier tells us how much the optimum improves if we relax the constraint.



$\mathcal{L} = xy - \alpha(2x + 2y - P)$. Setting the derivatives to zero:

$$\frac{\partial \mathcal{L}}{\partial x} = y - 2\alpha = 0, \qquad \frac{\partial \mathcal{L}}{\partial y} = x - 2\alpha = 0, \qquad 2x + 2y = P.$$

So $x = y = 2\alpha$: a square with $x = y = P/4$, and $\alpha = P/8$. Then $A^* = P^2/16$ and $\mathrm{d}A^*/\mathrm{d}P = P/8 = \alpha$.

[5 marks]

(b) Find the point $\boldsymbol{x}$ closest to the origin on the hyperplane $\boldsymbol{a}^\mathsf{T}\boldsymbol{x} = b$, by minimising $\|\boldsymbol{x}\|^2$ subject to this constraint. Hence show that the distance from the origin to the hyperplane is $|b|/\|\boldsymbol{a}\|$. (This is the geometry behind the margin of a support vector machine.)

$\mathcal{L} = \boldsymbol{x}^\mathsf{T}\boldsymbol{x} - \alpha(\boldsymbol{a}^\mathsf{T}\boldsymbol{x} - b)$, so $\nabla_{\boldsymbol{x}}\mathcal{L} = 2\boldsymbol{x} - \alpha\boldsymbol{a} = \boldsymbol{0}$ and $\boldsymbol{x} = \frac{\alpha}{2}\boldsymbol{a}$: the closest point is along the normal $\boldsymbol{a}$. The constraint gives $\frac{\alpha}{2}\|\boldsymbol{a}\|^2 = b$, so $\alpha = 2b/\|\boldsymbol{a}\|^2$ and

$$\boldsymbol{x}^* = \frac{b}{\|\boldsymbol{a}\|^2}\boldsymbol{a}, \qquad \|\boldsymbol{x}^*\| = \frac{|b|}{\|\boldsymbol{a}\|}.$$

[6 marks]

(c) Minimise $f(x,y) = (x-2)^2 + (y-1)^2$ subject to the inequality $x + y \leq 2$. Write the constraint in the form $g(x,y) \geq 0$, decide whether it is active, and find the solution and the value of α.

$g = 2 - x - y \geq 0$. The unconstrained minimum $(2,1)$ has $x + y = 3 > 2$, so it is not allowed and the constraint is active: the solution lies on $x + y = 2$. From $\nabla f = \alpha\nabla g$: $2(x-2) = -\alpha$ and $2(y-1) = -\alpha$, so $x - 2 = y - 1$. With $x + y = 2$ this gives $(x,y) = (1.5,0.5)$ and $\alpha = 1 \geq 0$, as KKT requires.

[3 marks]

(d) Repeat with the constraint $x + y \leq 4$.

Now the unconstrained minimum $(2,1)$ satisfies $x + y = 3 \leq 4$, so the constraint is inactive: $\alpha = 0$ and the solution is $(2,1)$ with $f = 0$.

[3 marks]

Answers

(e) Finally, take the *equality* constraint $x + y = 4$. Find the solution and α. Why would a negative α be impossible for the inequality constraint $x + y \leq 4$?

As in part (c), $x - 2 = y - 1$, and now $x + y = 4$, so $(x,y) = (2.5,1.5)$ with $f = 0.5$, and $2(x - 2) = -\alpha$ gives $\alpha = -1$. The equality forces us onto the line even though f would prefer to move *into* the region $x + y < 4$. A negative multiplier says exactly that: the constraint is pushing the wrong way, so as an inequality it would not be active. KKT's $\alpha \geq 0$ rules this case out and sends us back to part (d).

[3 marks]

End of question 2

Answers

• Do not write in this space •

3 In the lectures we saw that the linear programme (the *primal*)

$$\min_{\boldsymbol{x}} \boldsymbol{c}^{\mathsf{T}}\boldsymbol{x} \quad \text{subject to} \quad \mathbf{M}\boldsymbol{x} \geq \boldsymbol{b}, \quad \boldsymbol{x} \geq \mathbf{0}$$

has the *dual*

$$\max_{\boldsymbol{\alpha}} \boldsymbol{b}^{\mathsf{T}}\boldsymbol{\alpha} \quad \text{subject to} \quad \mathbf{M}^{\mathsf{T}}\boldsymbol{\alpha} \leq \boldsymbol{c}, \quad \boldsymbol{\alpha} \geq \mathbf{0}.$$

(a) Solve the linear programme: minimise $2x_1+3x_2$ subject to $x_1+x_2 \geq 4$, $x_1+3x_2 \geq 6$ and $x_1,x_2 \geq 0$. (The optimum of a linear programme is at a corner of the feasible region, so check the corners.) Write down the dual, solve it in the same way, and compare the two optimal values.

The corners of the primal feasible region are $(0,4)$, $(3,1)$ and $(6,0)$, with costs 12, 9 and 12. The minimum is 9 at $(3,1)$.

Here $\boldsymbol{c} = (2,3)^{\mathsf{T}}$, $\boldsymbol{b} = (4,6)^{\mathsf{T}}$ and $\mathbf{M} = \begin{pmatrix} 1 & 1 \\ 1 & 3 \end{pmatrix}$, so the dual is: maximise $4\alpha_1 + 6\alpha_2$ subject to $\alpha_1 + \alpha_2 \leq 2$, $\alpha_1 + 3\alpha_2 \leq 3$, $\alpha_1,\alpha_2 \geq 0$. Its corners $(0,0)$, $(2,0)$, $(1.5,0.5)$ and $(0,1)$ have values 0, 8, 9 and 6. The maximum is 9 at $(1.5,0.5)$: the same value as the primal.

[6 marks]

(b) Show that for *any* $\boldsymbol{x}$ satisfying the primal constraints and *any* $\boldsymbol{\alpha}$ satisfying the dual constraints, $\boldsymbol{b}^{\mathsf{T}}\boldsymbol{\alpha} \leq \boldsymbol{c}^{\mathsf{T}}\boldsymbol{x}$. Why is this useful?

Using $\boldsymbol{\alpha} \geq \mathbf{0}$ with $\mathbf{M}\boldsymbol{x} \geq \boldsymbol{b}$, and then $\boldsymbol{x} \geq \mathbf{0}$ with $\mathbf{M}^{\mathsf{T}}\boldsymbol{\alpha} \leq \boldsymbol{c}$:

$$\boldsymbol{b}^{\mathsf{T}}\boldsymbol{\alpha} \leq (\mathbf{M}\boldsymbol{x})^{\mathsf{T}}\boldsymbol{\alpha} = \boldsymbol{x}^{\mathsf{T}}\mathbf{M}^{\mathsf{T}}\boldsymbol{\alpha} \leq \boldsymbol{x}^{\mathsf{T}}\boldsymbol{c}.$$

This is *weak duality*. Any feasible dual point gives a lower bound on the primal optimum, so if we find feasible $\boldsymbol{x}$ and $\boldsymbol{\alpha}$ with equal values, as in part (a), both must be optimal.

[4 marks]

(c) If the primal has n variables and k constraints (not counting $\boldsymbol{x} \geq \mathbf{0}$), how many variables and constraints does the dual have? Which would you rather solve when $n = 10^6$ and $k = 10$?

The dual has k variables (one per primal constraint) and n constraints (one per primal variable). With $n = 10^6$ and $k = 10$ the dual has only 10 unknowns, which is usually far easier. The roles swap when $k \gg n$.

[3 marks]

ANSWERS

(d) Now consider the quadratic programme: minimise $\boldsymbol{x}^\mathsf{T}\boldsymbol{x}$ subject to $\mathbf{M}\boldsymbol{x} = \boldsymbol{b}$, where $\mathbf{M}$ is $k \times n$ with $k < n$. Form the Lagrangian, find $\boldsymbol{x}^*(\boldsymbol{\alpha})$, and show that the dual problem is

$$\max_{\boldsymbol{\alpha}} -\tfrac{1}{4}\boldsymbol{\alpha}^\mathsf{T}\mathbf{M}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha} + \boldsymbol{\alpha}^\mathsf{T}\boldsymbol{b}.$$

Solve it, and show that $\boldsymbol{x}^* = \mathbf{M}^\mathsf{T}(\mathbf{M}\mathbf{M}^\mathsf{T})^{-1}\boldsymbol{b}$. How big is the matrix we have to invert? Check that $k = 1$ recovers question 2(b).

$\mathcal{L} = \boldsymbol{x}^\mathsf{T}\boldsymbol{x} - \boldsymbol{\alpha}^\mathsf{T}(\mathbf{M}\boldsymbol{x} - \boldsymbol{b})$, so $\boldsymbol{\nabla}_{\boldsymbol{x}}\mathcal{L} = 2\boldsymbol{x} - \mathbf{M}^\mathsf{T}\boldsymbol{\alpha} = \mathbf{0}$ and $\boldsymbol{x}^*(\boldsymbol{\alpha}) = \frac{1}{2}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha}$. Substituting,

$$\mathcal{L} = \tfrac{1}{4}\boldsymbol{\alpha}^\mathsf{T}\mathbf{M}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha} - \tfrac{1}{2}\boldsymbol{\alpha}^\mathsf{T}\mathbf{M}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha} + \boldsymbol{\alpha}^\mathsf{T}\boldsymbol{b} = -\tfrac{1}{4}\boldsymbol{\alpha}^\mathsf{T}\mathbf{M}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha} + \boldsymbol{\alpha}^\mathsf{T}\boldsymbol{b}.$$

Setting its gradient $-\frac{1}{2}\mathbf{M}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha} + \boldsymbol{b}$ to zero gives $\boldsymbol{\alpha} = 2(\mathbf{M}\mathbf{M}^\mathsf{T})^{-1}\boldsymbol{b}$, so $\boldsymbol{x}^* = \mathbf{M}^\mathsf{T}(\mathbf{M}\mathbf{M}^\mathsf{T})^{-1}\boldsymbol{b}$. We invert a $k \times k$ matrix rather than work with n unknowns. This is the minimum-norm solution of an under-constrained system from the mappings lecture. With $k = 1$ and $\mathbf{M} = \boldsymbol{a}^\mathsf{T}$ it gives $\boldsymbol{x}^* = b\boldsymbol{a}/\|\boldsymbol{a}\|^2$, as in question 2(b).

[7 marks]

End of question 3

Answers

• Do not write in this space •

4 In this question you check some of these results in Python (or any other language). The following may be useful.

`import numpy as np`	import the numpy module
`H = np.diag([100.0, 1.0])`	a diagonal matrix
`np.array([1.0, 1.0])`	a vector
`H @ w`	matrix–vector multiplication
`np.linalg.norm(w)`	the length $\|\boldsymbol{w}\|$
`rng = np.random.default_rng(1)`	a random number generator with a fixed seed, so results can be repeated
`rng.integers(0, m, size=10)`	10 random indices, for a mini-batch

(a) Run gradient descent on $f(\boldsymbol{w}) = \frac{1}{2}(100w_1^2 + w_2^2)$ from $\boldsymbol{w} = (1,1)^\mathsf{T}$. For $r = 0.005$, $r = 2/101$ and $r = 0.0201$, count the steps until $\|\boldsymbol{w}\| < 10^{-6}$ (or report that it diverges). Compare with question 1.

```
H = np.diag([100.0, 1.0])
def run(r, gamma=0.0):
    w = np.array([1.0, 1.0]); v = np.zeros(2)
    for t in range(1, 100001):
        v = gamma*v - r*(H @ w)
        w = w + v
        if np.linalg.norm(w) < 1e-6: return t
        if np.linalg.norm(w) > 1e6: return 'diverged'
for r in [0.005, 2/101, 0.0201]: print(r, run(r))
```

This gives 2757 steps, 709 steps, and divergence. With $r = 2/101$ both components shrink by $99/101$ per step, and starting from $\|\boldsymbol{w}\| = \sqrt{2}$ that needs about 709 steps, close to the 690 of question 1(d). At $r = 0.0201 > 2/100$ the steep component is multiplied by -1.01 each step and grows.

[5 marks]

(b) Add momentum, $\boldsymbol{v} \leftarrow \gamma\boldsymbol{v} - r\nabla f(\boldsymbol{w})$, $\boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{v}$. Count the steps for $\gamma = 0.9$, $r = 0.01$, and for the tuned values $r = 4/121$, $\gamma = (9/11)^2$. Compare with plain gradient descent.

Using `run` from part (a): `run(0.01, 0.9)` takes 264 steps and `run(4/121, (9/11)**2)` takes 95, against 709 for the best plain gradient descent. With tuned momentum the error falls by about $9/11 = (\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$ per step rather than $99/101 = (\kappa-1)/(\kappa+1)$: the steps needed scale as $\sqrt{\kappa} = 10$ rather than $\kappa = 100$.

[5 marks]

(c) Code Newton's method for $f(w) = \mathrm{e}^w - 2w$ (question 1(e)) and print $\log_{10}|w_t - \ln 2|$ for the first six steps.

```
w = 0.0
for t in range(6):
    print(t, np.log10(abs(w - np.log(2))))
    w = w - 1 + 2*np.exp(-w)
```

The logarithm of the error goes roughly -0.2, -0.5, -1.4, -3.0, -6.4, -13.1: it roughly doubles each step, the signature of quadratic convergence.

[4 marks]

(d) Generate 1000 data points with x uniform on $[-1,1]$ and $y = 2x+1+0.5\epsilon$, where ϵ is a standard normal deviate. Fit $y \approx w_1 x + w_2$ by mini-batch SGD on the mean squared error, with batches of 10, $r = 0.1$ for 2000 steps and then $r = 0.01$ for 2000 steps. Record the loss on all the data after every step. Compare its average over the last 200 steps of each phase with the best possible loss, found by least squares. What does lowering r do?

```
rng = np.random.default_rng(1)
x = rng.uniform(-1, 1, 1000)
y = 2*x + 1 + 0.5*rng.normal(size=1000)
X = np.column_stack([x, np.ones(1000)])
w = np.zeros(2); r = 0.1; L = []
for t in range(4000):
    if t == 2000: r = 0.01
    b = rng.integers(0, 1000, size=10)
    w = w - r*2*X[b].T @ (X[b] @ w - y[b])/10
    L.append(np.mean((X @ w - y)**2))
```

The best possible loss is about 0.264. With $r = 0.1$ the loss hovers about 0.006 above it, and after the drop to $r = 0.01$ only about 0.0003 above (the exact numbers depend on the random seed). Mini-batch noise keeps SGD jiggling around the minimum, and the size of the jiggle shrinks with r. That is why learning-rate schedules lower r towards the end of training.

[6 marks]

End of question 4

END OF PAPER